

Operaciones CRUD y gestión de transacciones

3. Operaciones CRUD y gestión de transacciones

Operaciones CRUD y gestión de transacciones

Ya sabes por qué hacen falta conectores y cómo se declara la estructura de una base de datos desde Java. Toca el paso siguiente: cómo se modifican y consultan los datos desde una aplicación real. Vas a construir el CRUD de Libro igual que lo harías tú solo — empezando por la versión más simple, tropezando con sus problemas, y resolviéndolos uno a uno.

Qué es CRUD

CRUD son las siglas de las cuatro operaciones universales sobre datos: Create, Read, Update, Delete. HTTP expresa esas mismas intenciones sobre recursos:

CRUD	SQL	HTTP
Create	INSERT	POST
Read	SELECT	GET
Update	UPDATE	PUT (reemplazo completo); PATCH (parcial)
Delete	DELETE	DELETE

REST está pensado precisamente para modelar estas operaciones sobre recursos con los verbos que HTTP ya trae, en vez de inventar una ruta distinta para cada acción (/libros/crear sería el estilo antiguo, no REST).

La primera pieza: el repository

El repository es quien de verdad habla con la base de datos. Con JDBC "a pelo" escribir un repository significa escribir tú mismo cada consulta SQL. Con Spring Data JPA ese trabajo desaparece casi por completo: declaras una interfaz vacía, y Spring te da una implementación real ya funcionando.

```
public interface LibroRepository extends JpaRepository<Libro, Long> {}
```

Qué significa `extends JpaRepository<Libro, Long>`

`JpaRepository<T, ID>` es una interfaz que trae Spring Data JPA, con dos parámetros de tipo entre `< >` — como los de `List<String>` o `Optional<Libro>`, que ya conoces:

```
JpaRepository<Libro, Long>  
              T      ID
```

- T: la entidad que gestiona este repository — aquí, `Libro`.
- ID: el tipo de la clave primaria de esa entidad — el mismo tipo del campo `@Id` (en este curso, siempre `Long`).

Qué significa `extends JpaRepository` (cont.)

`JpaRepository` no está vacía: por dentro ya trae declarados decenas de métodos — `findAll()`, `findById(id)`, `save(entidad)`, `deleteById(id)`... `LibroRepository` hereda todos esos métodos automáticamente, ya especializados para `Libro`, aunque no escribas ni una línea dentro de las llaves.

¿Cómo puede funcionar una interfaz sin implementación? Spring Data JPA, al arrancar la aplicación, genera una clase real que implementa `LibroRepository` por completo y la registra como un bean, igual que los beans `@Service`/`@Repository` que ya conoces. Tú nunca ves esa clase generada — solo inyectas la interfaz.

Lo que ya tienes disponible, sin escribir nada

Método	Qué hace
List<Libro> findAll()	Devuelve todas las filas de la tabla libro.
Optional<Libro> findById(Long id)	Busca una fila por su clave primaria. Puede no encontrar nada.
Libro save(Libro libro)	Inserta si el id es null, o actualiza si ya existe — sirve para crear y modificar.
boolean existsById(Long id)	Comprueba si existe una fila, sin cargar la entidad entera.
void deleteById(Long id)	Borra la fila con ese id.
long count()	Cuenta cuántas filas hay en la tabla.

El Optional de findById

`findById` no devuelve un `Libro`, devuelve un `Optional<Libro>`. Un `Optional` es un envoltorio que representa "puede que haya un valor, puede que no" — te obliga a decidir explícitamente qué hacer en el caso "no encontrado":

```
libroRepository.findById(id)
    .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Libro
no encontrado"));
```

`orElseThrow(...)` dice, en una línea: "si el `Optional` trae un libro, dámelo; si viene vacío, lanza esta excepción en su lugar".

Y si necesitas una consulta que no viene de fábrica

No hace falta escribir SQL a mano ni implementar nada: declaras el método en la interfaz, sin cuerpo, siguiendo un patrón de nombre:

```
public interface LibroRepository extends JpaRepository<Libro, Long> {  
    List<Libro> findByEditorialId(Long editorialId);  
}
```

Spring Data JPA lee el nombre del método, lo descompone (find + By + EditorialId) y genera la consulta él solo. El resultado equivale a:

```
SELECT * FROM libro WHERE editorial_id = ?
```

El patrón de nombres, combinable

Patrón	Ejemplo	Equivale a
findByX	findByTitulo(String titulo)	WHERE titulo = ?
findByXAndY	findByTituloAndPrecioLessThan(...)	WHERE titulo = ? AND precio < ?
findByXOrderByY	findByEditorialIdOrderByPrecioDesc(...)	WHERE editorial_id = ? ORDER BY precio DESC
existsByX	existsByTitulo(String titulo)	igual que findByX, devuelve boolean
countByX	countByEditorialId(Long id)	igual, pero devuelve long

Cuidado con los límites de esta convención: es cómoda para consultas simples y fijas. Para filtros que cambian según el caso están las Specifications y @Query con JPQL, que verás más adelante en este tema.

El repository que vas a usar en este apartado

El CRUD de este apartado no necesita ninguna consulta personalizada todavía — `findAll/findById/save/deleteById` bastan:

```
public interface LibroRepository extends JpaRepository<Libro, Long> {}  
public interface EditorialRepository extends JpaRepository<Editorial, Long> {}
```

Con las dos interfaces declaradas, tienes las tres capas listas: Controller recibe la petición HTTP, Service decide qué hacer, Repository habla con la base de datos.

Primer intento: leer y devolver la entidad tal cual

La forma más directa: el controller le pide al service el libro, el service se lo pide al repository, y todo el mundo se pasa el mismo objeto Libro.

```
@Service
@RequiredArgsConstructor
public class LibroService {
    private final LibroRepository libroRepository;

    @Transactional(readOnly = true)
    public Libro findById(Long id) {
        return libroRepository.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Libro no
encontrado"));
    }
}
```

Primer intento (cont.)

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService libroService;

    @GetMapping("/{id}")
    public ResponseEntity<Libro> getById(@PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
}
```

Primer intento (cont.)

`@Transactional(readOnly = true)` indica que este método trabaja con la base de datos únicamente para consultar, sin intención de modificarla. Más adelante verás qué significa exactamente una transacción y qué aporta `readOnly`.

El controller no toca el repository directamente, solo habla con el service; el service es quien de verdad llama a `libroRepository.findById(...)`. Ese orden (Controller → Service → Repository) no cambia en todo lo que viene a continuación.

Primer intento (cont.)

Compila, funciona, y GET /api/v1/libros/3 te devuelve algo así:

```
{
  "id": 3,
  "titulo": "El nombre del viento",
  "precio": 19.95,
  "coste": 8.40,
  "fechaPublicacion": "2007-03-27",
  "editorial": { "id": 1, "nombre": "Plaza & Janés", "hibernateLazyInitializer": {} }
}
```

El problema: la entidad expone más de lo que quieres

Campo	Por qué es un problema
coste	El precio de compra al proveedor — un dato interno que jamás debería llegar a un cliente de la API. Cualquier campo <code>private</code> de la entidad viaja tal cual al JSON, lo quieras exponer o no.
hibernateLazyInitializer	Un detalle interno de cómo Hibernate representa la relación <code>@ManyToOne</code> con carga lazy, filtrado sin querer al convertir el objeto a JSON.

La entidad `Libro` está diseñada para que Hibernate la entienda, no para que la vea un cliente HTTP. Son dos propósitos distintos, y una sola clase no puede servir bien a los dos a la vez.

La solución: DTOs

Un DTO (Data Transfer Object) es un objeto hecho a medida de lo que sale (o entra) de tu aplicación. Antes de escribirlo, una construcción que encaja perfectamente: el record (desde Java 16), que declara en una línea una clase inmutable:

```
record Punto(int x, int y) {}
```

El compilador genera automáticamente un constructor, un getter por campo (`punto.x()`, no `punto.getX()`) y `equals()/hashCode()/toString()`. Todos los campos son final — no hay setters. Esa inmutabilidad es justo lo que necesitas para un DTO.

La solución: DTOs (cont.)

Un record NO es lo mismo que una @Entity: Libro/Editorial siguen siendo clases normales, con @Getter/@Setter de Lombok, porque necesitan ser mutables (Hibernate las modifica) y tener un identificador gestionado por el framework. Si ves record en una clase, es un DTO; si ves @Entity, es una entidad persistente.

El DTO de salida

```
public record EditorialDTO(Long id, String nombre) {}

public record LibroResponseDTO(
    Long id,
    String titulo,
    BigDecimal precio,
    LocalDate fechaPublicacion,
    EditorialDTO editorial
) {}
```

Ni rastro de coste, ni de los detalles internos de Hibernate — solo lo que quieres que vea un cliente.

findById traduce con mapToDTO

```
@Transactional(readOnly = true)
public LibroResponseDTO findById(Long id) {
    Libro libro = libroRepository.findById(id)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Libro no encontrado"));
    return mapToDTO(libro);
}

private LibroResponseDTO mapToDTO(Libro libro) {
    EditorialDTO editorialDTO = new EditorialDTO(libro.getEditorial().getId(),
libro.getEditorial().getNombre());
    return new LibroResponseDTO(
        libro.getId(), libro.getTitulo(), libro.getPrecio(), libro.getFechaPublicacion(),
editorialDTO
    );
}
```

mapToDTO, campo a campo

mapToDTO es private — el controller no lo llama nunca, es solo cómo LibroService traduce cada Libro al DTO que ve el cliente:

Campo de LibroResponseDTO	De dónde sale
id	libro.getId()
titulo	libro.getTitulo()
precio	libro.getPrecio()
fechaPublicacion	libro.getFechaPublicacion()
editorial	Un EditorialDTO nuevo, construido a partir de libro.getEditorial()
(no aparece)	coste no tiene fila en esta tabla: no se copia, por eso desaparece del JSON

El controller cambia una sola palabra

A partir de aquí, cualquier método de LibroService que necesite devolver un libro (findAll, create, update) va a llamar a este mismo mapToDTO en vez de repetir la traducción cada vez. El controller solo cambia el tipo que envuelve ResponseEntity:

```
@GetMapping("/{id}")  
public ResponseEntity<LibroResponseDTO> getById(@PathVariable Long id) {  
    return ResponseEntity.ok(libroService.findById(id));  
}
```

Ahora la escritura: create()

Para crear un libro necesitas un DTO de entrada — no vale reutilizar LibroResponseDTO: al crear no tiene sentido pedir un id. Un primer intento, sin pensarlo mucho:

```
public record LibroCreateDTO(String titulo, BigDecimal precio, LocalDate fechaPublicacion, Long editorialId) {}
```

```
@Transactional
public LibroResponseDTO create(LibroCreateDTO dto) {
    Editorial editorial = editorialRepository.findById(dto.editorialId())
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Editorial no encontrada"));
    Libro libro = new Libro();
    libro.setTitulo(dto.titulo());
    libro.setPrecio(dto.precio());
    libro.setFechaPublicacion(dto.fechaPublicacion());
    libro.setEditorial(editorial);
    return mapToDTO(libroRepository.save(libro));
}
```

El problema de los datos basura

Funciona... con datos razonables. Pero nada impide mandar {"titulo": "", "precio": -5, "fechaPublicacion": "2099-01-01", "editorialId": -1}. Tu aplicación lo aceptaría igual, porque create() no comprueba nada de eso. La solución: anotaciones de Bean Validation sobre el propio DTO.

Falta una dependencia en el pom.xml: spring-boot-starter-webmvc no trae Bean Validation incluido. Añade spring-boot-starter-validation, que trae Hibernate Validator — sin ella, @NotBlank y compañía ni siquiera están en el classpath.

El DTO de entrada, con Bean Validation

```
public record LibroCreatedDTO(  
    @NotBlank String titulo,  
    @NotNull @PositiveOrZero BigDecimal precio,  
    @NotNull @PastOrPresent LocalDate fechaPublicacion,  
    @NotNull @Positive Long editorialId  
) {}
```

La familia de Bean Validation

Anotación	Qué exige	En este DTO
@NotNull	El valor no puede ser null.	precio, fechaPublicacion, editorialId
@NotBlank	Además de no null, si es String no puede estar vacío ni ser solo espacios.	titulo
@NotEmpty	Igual que @NotBlank, pero para colecciones/arrays.	—
@Size(min=, max=)	La longitud (String o colección) debe caer en ese rango.	—
@Positive / @PositiveOrZero	El número debe ser mayor que cero / mayor o igual.	editorialId / precio
@Negative / @NegativeOrZero	Lo mismo, en negativo.	—

La familia de Bean Validation (cont.)

Anotación	Qué exige	En este DTO
@Min(valor) / @Max(valor)	El número no puede bajar de / superar ese valor.	—
@Past / @PastOrPresent	La fecha debe ser anterior a ahora / anterior o igual.	fechaPublicacion
@Future / @FutureOrPresent	La fecha debe ser posterior a ahora / posterior o igual.	—
@Email	El String debe tener forma de dirección de correo.	—
@Pattern(regexp = ...)	El String debe coincidir con la expresión regular dada.	—

Todas se pueden combinar sobre el mismo campo, como @NotNull @PositiveOrZero en precio.

@Valid activa las comprobaciones

```
@PostMapping
public ResponseEntity<LibroResponseDTO> create(@Valid @RequestBody
LibroCreateDTO dto) {
    return
    ResponseEntity.status(HttpStatus.CREATED).body(libroService.create(dto));
}
```

@RequestBody convierte el JSON en un LibroCreateDTO; @Valid activa de verdad las comprobaciones. Si algún campo incumple su restricción, Spring rechaza la petición con un 400 Bad Request antes de que tu código llegue a ejecutarse.

update() y delete(): el recurso que no existe

update() reutiliza LibroCreateDTO y sigue un patrón parecido, con un matiz: primero tiene que cargar el libro que ya existe.

```
@Transactional
public LibroResponseDTO update(Long id, LibroCreateDTO dto) {
    Libro libro = libroRepository.findById(id)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Libro no encontrado"));
    Editorial editorial = editorialRepository.findById(dto.editorialId())
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Editorial no
    encontrada"));
    libro.setTitulo(dto.titulo());
    libro.setPrecio(dto.precio());
    libro.setFechaPublicacion(dto.fechaPublicacion());
    libro.setEditorial(editorial);
    return mapToDTO(libroRepository.save(libro));
}
```

update() y delete() (cont.)

Sin la comprobación de `orElseThrow` podrías terminar utilizando un valor `null`: el código compilaría, pero fallaría durante la ejecución. `delete()` no tiene nada que cargar, así que usa la comprobación más ligera:

```
@Transactional
public void delete(Long id) {
    if (!libroRepository.existsById(id)) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Libro no
    encontrado");
    }
    libroRepository.deleteById(id);
}
```

update() y delete() (cont.)

`findAll()` no tiene ninguno de estos problemas: una lista puede estar vacía sin que eso sea un error. Como solo consulta datos, también utiliza `@Transactional(readOnly = true)` y transforma cada entidad en su DTO correspondiente.

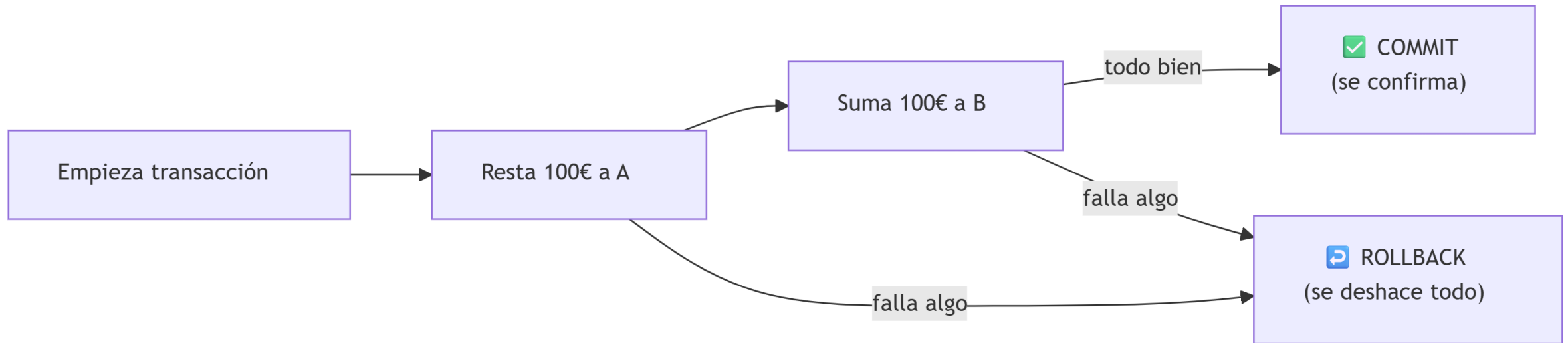
Qué garantiza que las escrituras no queden a medias

Mira otra vez `update()`: hace dos lecturas y una escritura. ¿Qué pasaría si el proceso se cayera justo después de guardar el libro, pero antes de que la operación completa terminara de confirmarse?

Es el mismo problema, en miniatura, que una transferencia bancaria: restar 100€ de una cuenta y sumar 100€ a otra son dos operaciones — si el programa falla justo después de restar, el dinero desaparece. Una transacción agrupa operaciones para que se comporten como una sola unidad: todo o nada.

Commit o rollback: todo o nada

Si todas las operaciones terminan bien, se confirman de golpe (commit); si cualquiera falla a mitad, se deshace todo lo hecho hasta ese punto (rollback), como si nunca hubiera pasado nada:



Las propiedades ACID

Propiedad	Qué garantiza
Atomicidad	Se ejecuta entera, o no se ejecuta nada de ella.
Consistencia	La base de datos pasa de un estado válido a otro, nunca a uno a medias.
Aislamiento	Dos transacciones que ocurren al mismo tiempo no se interfieren entre sí.
Durabilidad	Una vez confirmada (commit), sobrevive aunque el sistema se caiga justo después.

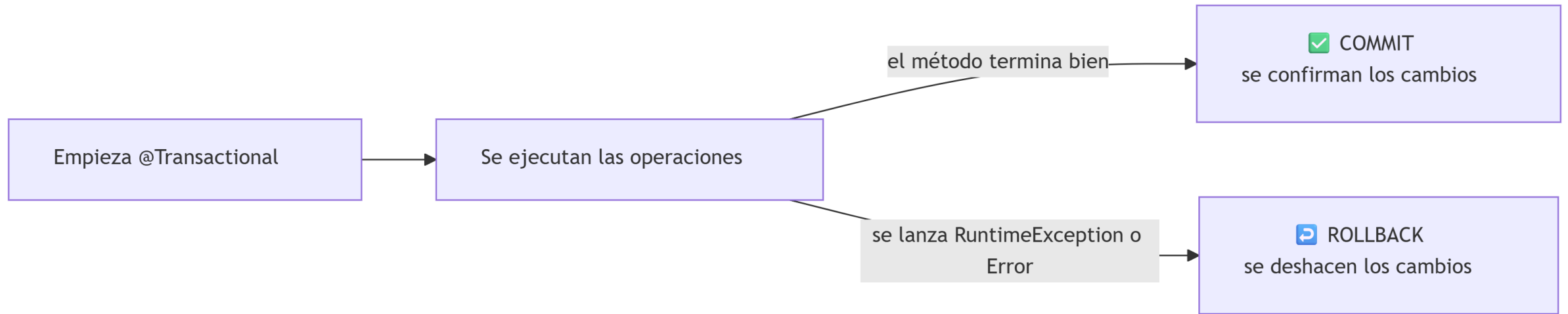
Todo esto lo gestiona Spring por ti

`update()` y `delete()` llevan `@Transactional`; `findById/findAll` llevan `@Transactional(readOnly = true)`. Eso es todo lo que hace falta — tú no escribes `connection.commit()` ni `connection.rollback()` en ningún sitio. Por debajo, Spring:

- Abre la transacción antes de ejecutar el método.
- Hace commit automáticamente si el método termina correctamente.
- Hace rollback si el método termina con una excepción no comprobada (`RuntimeException`) o un error grave.

El ciclo de @Transactional

Las excepciones que usas en este CRUD, como `ResponseStatusException`, pertenecen al grupo que dispara el rollback automático:



Excepciones comprobadas: el caso especial

Java también tiene excepciones comprobadas, que obligan a capturarlas o declararlas en la firma del método. Estas no provocan un rollback automático con la configuración habitual de Spring. Cuando necesitas que también deshagan la transacción, puedes indicarlo expresamente:

```
@Transactional(rollbackFor = IOException.class)
public void importarLibros() throws IOException {
    // ...
}
```

Contraste con lo que viene después: en el siguiente apartado (JDBC puro) gestionarás una transacción a mano, desactivando el autocommit y decidiendo cuándo ejecutar commit() o rollback().

Transacciones de solo lectura

```
@Transactional(readOnly = true)
public LibroResponseDTO findById(Long id) {
    // ...
}
```

Esta opción indica a Spring y a Hibernate que durante el método solo se espera consultar información. Eso permite optimizaciones y deja clara la intención del código. Por eso, en este CRUD sigues esta regla:

Operación	Anotación
Consultar datos	@Transactional(readOnly = true)
Crear, actualizar o eliminar	@Transactional

Las piezas juntas: el LibroController completo

El controller completo es la traducción directa de la tabla CRUD↔HTTP del principio — cada método delega en su equivalente de LibroService:

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService libroService;

    @GetMapping
    public ResponseEntity<List<LibroResponseDTO>> getAll() {
        return ResponseEntity.ok(libroService.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<LibroResponseDTO> getById(@PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
    // ...continúa: create, update, delete
```

El LibroController completo (cont.)

```
@PostMapping
public ResponseEntity<LibroResponseDTO> create(@Valid @RequestBody LibroCreateDTO dto) {
    return ResponseEntity.status(HttpStatus.CREATED).body(libroService.create(dto));
}

@PutMapping("/{id}")
public ResponseEntity<LibroResponseDTO> update(@PathVariable Long id, @Valid @RequestBody
LibroCreateDTO dto) {
    return ResponseEntity.ok(libroService.update(id, dto));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    libroService.delete(id);
    return ResponseEntity.noContent().build();
}
}
```

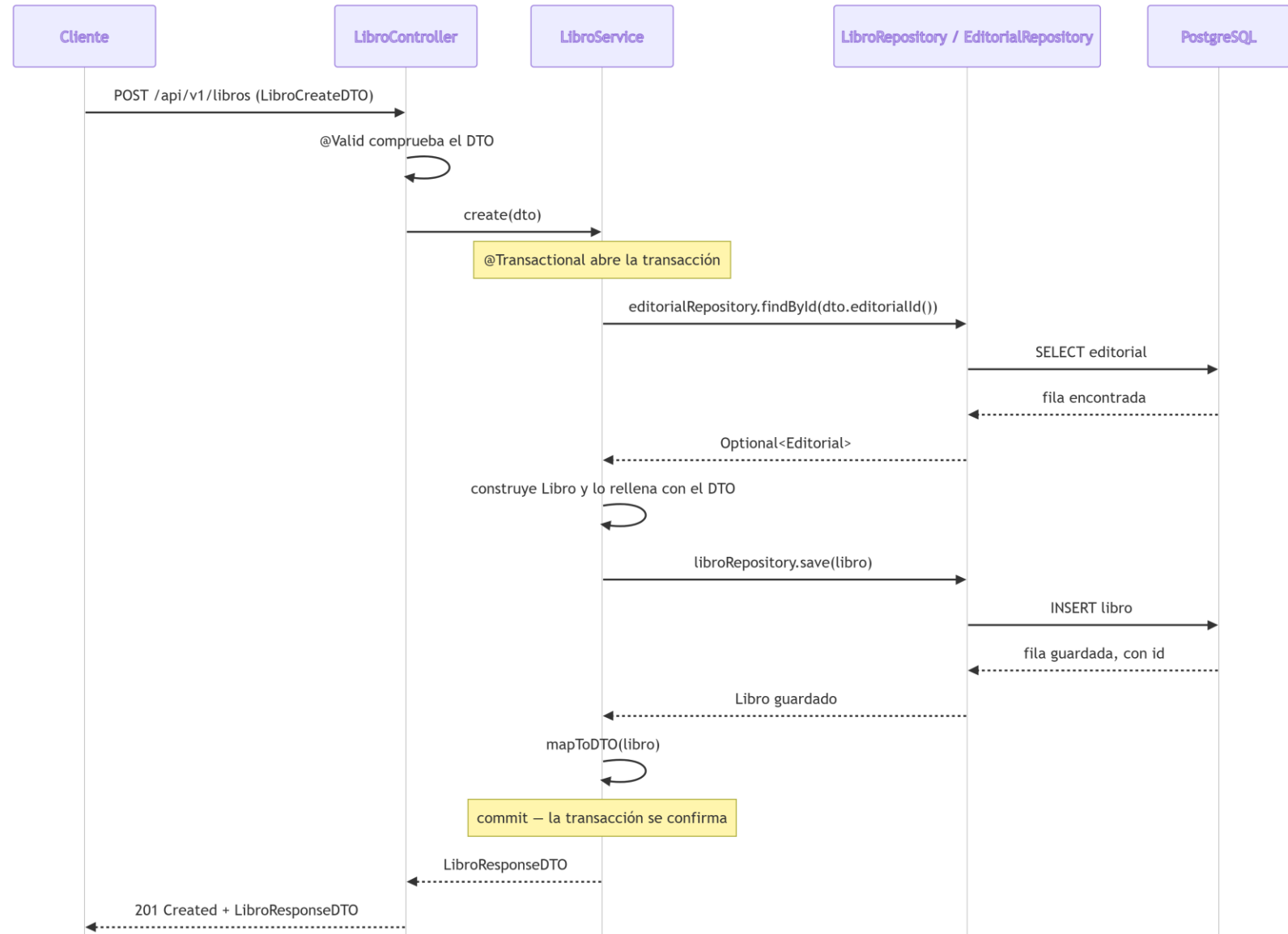

El LibroController completo (cont.)

Cada método usa exactamente el código de estado que anticipaba la tabla del principio: create responde 201, update responde 200, delete responde 204 sin cuerpo. Y se repite la misma regla de todo el apartado: el controller no habla nunca con el repository, ni construye nunca SQL — solo traduce entre HTTP y una llamada a LibroService.

Con todo esto ya puedes abordar la Actividad 1.2: construir, guiado, un CRUD completo como este en tu propio proyecto.

El viaje completo, de principio a fin

Todas las piezas del apartado, juntas de una vez: así es el recorrido real de un POST /api/v1/libros, desde que llega la petición hasta que el cliente recibe el 201:



Ideas clave

- CRUD = Create/Read/Update/Delete. En HTTP, Read puede expresarse con GET, y Update con PUT o PATCH.
- Un repository (interface XRepository extends JpaRepository<X, Long>) te da findAll/findById/save/existsById/deleteById sin escribir SQL ni implementación propia.
- Puedes declarar tus propias consultas sin SQL, con un método sin cuerpo cuyo nombre siga la convención (findByEditorialId...) — para filtros dinámicos están las Specifications y @Query.
- Devolver la entidad JPA tal cual expone campos internos que no deberías publicar — por eso se convierte a un DTO antes de salir.
- Un record de Java declara en una línea una clase inmutable — encaja con los DTOs porque nunca cambian tras crearse.

Ideas clave (cont.)

- Las anotaciones de Bean Validation (`@NotBlank`, `@PositiveOrZero...`) sobre el DTO de entrada, con `@Valid` en el controller, rechazan datos basura con un 400 antes de ejecutar tu código.
- El caso "recurso no encontrado" se resuelve con `orElseThrow(...)/existsById(...)` lanzando `ResponseStatusException(HttpStatus.NOT_FOUND, ...)`.
- Una transacción agrupa operaciones como una unidad todo-o-nada: commit si todo va bien, rollback si algo falla. Las propiedades ACID la garantizan.
- `@Transactional` lo gestiona automáticamente; `@Transactional(readOnly = true)` marca las operaciones de solo lectura.
- En todo el apartado, el flujo es siempre el mismo: Controller → Service → Repository, cada uno hablando solo con el siguiente.